

TSP – Closest Edge Insertion Heuristic

Carli Crump
Computer Science and Engineering
Speed School of Engineering
University of Louisville, USA
cacrum03@louisville.edu

1. Introduction

In this project, Closest Edge Insertion algorithm was implemented to approximate a solution for the Traveling Salesperson Problem (TSP). The TSP problem is a NP-complete problem where a salesperson is given a list of cities, and the goal is to find the shortest possible path that visits each city one time and ends at the starting city. This project explores a greedy heuristic approach where each city is inserted to an existing edge that is already in the path. The purpose of this project was to explore heuristic insertion methods for the TSP and compare the performance to previous methods: brute-force search, breadth-first search, and depth-first search.

2. Approach

The Closest Edge Insertion algorithm begins by selecting two initial cities to form the starting tour. In this project I chose the two cities that were closest to the x-axis. If multiple cities were equally close, then the pair with the shortest Euclidean distance between them were selected. I chose this approach to try to reduce the starting tour length and provide a stable base for further insertions. After initializing the two starting cities, the path is then built incrementally. For each city that was not in the path yet, the algorithm computes the distance to every edge of the current tour using the point-to-segment distance formula. The minimum distance and the edge it's closest to is recorded. The next city inserted is then selected by the smallest distance to any edge and it is placed in the path between the two cities that make up the edge.

The process is repeated until all cities are included in the path and the path is closed by connecting the last city back to the first. The advantage of this algorithm is that instead of attaching each city to the closest city, it is considering the overall structure of the path, which often leads to shorter routes. The point-to-segment distance calculation in the project ensures that each new city is inserted in the location that minimally increases the total length of the path.

3. Results

3.1 Data

Two datasets were used to evaluate the Closest Edge Insertion algorithm. *Random30.tsp* (figure 1) is the first dataset and contains 30 cities with randomly generated two-dimensional coordinates. The second, *Random40.tsp* (figure 2), contains 40 cities generated the same way. Both datasets provide each city's (x, y) position which is parsed into a list of coordinate pairs that are used to calculate the Euclidean distances between them during the algorithm.

3.2 Results

Dataset	Starting Cities	Path	Distance
Random30.tsp	20, 6	20, 8, 10, 15, 30, 1, 24, 7, 21, 3, 28, 19, 23, 17, 16, 11, 22, 27, 9, 5, 26, 14, 2, 12, 18, 4, 29, 13, 25, 6	511.66
Random40.tsp	20, 6	20, 38, 8, 40, 10, 16, 39, 15, 30, 1, 24, 7, 17, 23, 21, 3, 19, 32, 28, 36, 34, 37, 11, 12, 18, 2, 14, 22, 27, 9, 5, 26, 35, 4, 31, 33, 29, 13, 25, 6	618.89

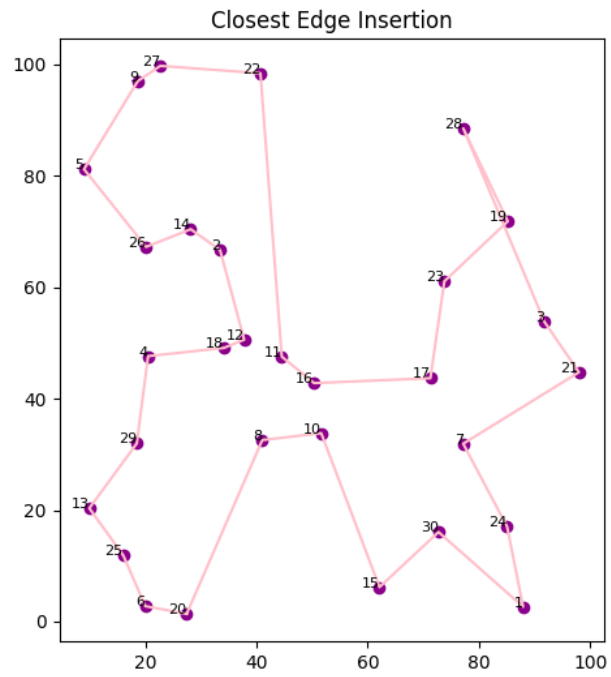


Figure 1

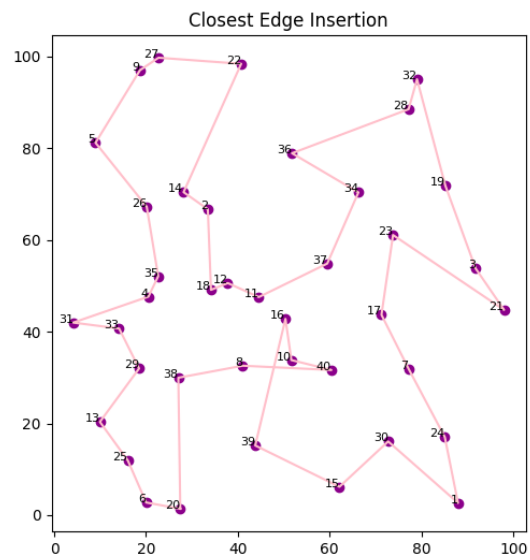


Figure 2

4. Discussion

The approach of starting with the two cities closest to the x-axis provides a short and balanced initial edge, which helps guide the construction of a more compact path. Other methods for picking the initial cities such as choosing the two closest cities lead to a longer path. Once the initial cities were chosen, the algorithm proceeds by evaluating every possible edge and determining the best position to insert the next city. It selects the city whose shortest distance to any existing tour edge is minimal to have each insertion add the shortest additional distance as possible.

Compared to other approaches for solving TSP, closest edge insertion is more flexible, since it allows cities to be inserted into the most favorable location in the path, not just at the end. The brute-force algorithm had to evaluate all possible paths which would guarantee the optimal solution, but it is only feasible for smaller datasets of cities. BFS explores all possible levels of paths which ensures the optimal solution but similar to brute-force, becomes impractical for larger datasets. DFS explores the whole path before backtracking and uses less memory than BFS but like the others, experiences the same exponential runtime complexity. DFS is also not guaranteed to find the most optimal path. Closest edge insertion finds solutions much faster than the previous three algorithms. While it does optimize path distances, it is not guaranteed to find the most optimal path unless every possible starting city is attempted.

5. References

Weisstein, Eric W. "Point-Line Distance--2-Dimensional." *Mathworld.wolfram.com*, mathworld.wolfram.com/Point-LineDistance2-Dimensional.html.

Python Software Foundation. "Tkinter — Python Interface to Tcl/Tk — Python 3.7.2 Documentation." *Python.org*, 2019, docs.python.org/3/library/tkinter.html.